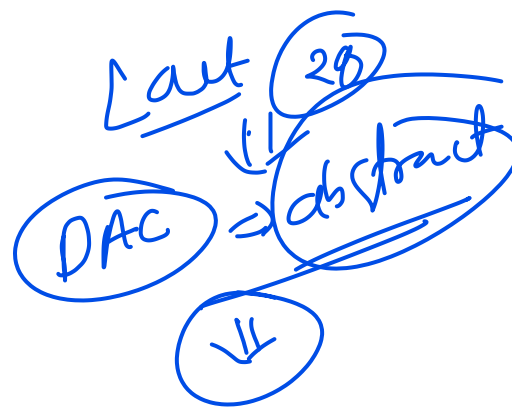


15x



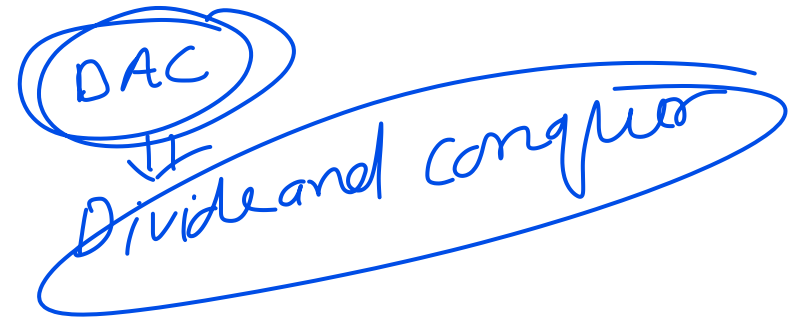
Lec_29

Applications or DAC Algorithms



✓ In GATE Syllabus:

- ✓ 1. Finding min and max together
- ✓ 2. Power of an element
- ✓ 3. Binary search
- ✓ 4. Merge sort
- ✓ 5. Quick Sort
- ✓ 6. Counting Number of inversions
- ✓ 7. Selection procedure
- ✓ 8. Strassen's Matrix Multiplication
- ✓ 9. Contiguous Maximum Subarray Sum



⇒ 4th
apph-
merge, tree

✓ 1.) Finding Max and Min Together

• **Without DAC:**

• Example:

input: an array of n elements
output: $\max^m$ and $\min^m$ element of the array.

Ex: $a = [10, 2, 5, 6, 3, 1, 27, 9]$
 $n = 8$, $\max^m = 27$, $\min^m = 1$

$\max = a[0] = 10$
 $\min = a[0] = 10$

for ($i = 1$; $i \leq n-1$; $i++$)
{
 if ($a[i] > \max$)
 $\max = a[i]$
}

10, 2, 5, 6, 3, 1, 27, 9
↑ ↑ ↑ ↑ ↑
 $\max = 10$, $\min = 10$
 $\max = 27$, $\min = 1$

$\max = 27$
 $\min = 1$

alg 0
without
DAC

$\{$
 $\quad \text{else if } (a[i] < \text{min})$
 $\quad \quad \text{min} = a[i]$
 $\}$

$\rightarrow$ no of comparisons of the elements of array
 1.) $(n-1)$ comparisons : Best case
 2.) $2(n-1)$ " : worst case

$\text{max} = 10, \text{min} = 10$
 $\text{for } (i = 1; i \leq n-1; i++)$

$\{$
 $\quad \text{if } (a[i] > \text{max})$

$\quad \{ \text{max} = a[i] \}$

$\quad \text{else}$

$\quad \{ \text{if } (a[i] < \text{min})$

$\quad \quad \text{min} = a[i]$

array in increasing order

Best case no. compars = $n-1$

worst case " " = $2(n-1)$

Avg case = $\frac{(n-1)}{2} + \frac{2(n-1)}{2}$

$= \frac{n-1}{2} + (n-1)$

$= \frac{3(n-1)}{2} = 1.5(n-1)$

$\uparrow B \quad \quad \uparrow A \quad \quad \uparrow C$
 $n-1, 1.5(n-1), 2(n-1)$

$10, 13, 15, 18, 17, 23$
 $\text{max} = 10, \text{min} = 10$
 $a[i] > \text{max}$

Best case: if array is in increasing order
↓
no. of comp = $n-1$

~~Algorithm:~~

if array is sorted (all)
in decreasing order
↓
 $2(n-1)$
time

10, 8, 4, 3, 1, 0
min = 10, max = 10
↓
always OK

$a[i] > \text{max}$

without DRC min-max

Time complexity = $\Theta(n)$, $O(n)$, $\sqrt{n}$

no. of comparisons = B

A	B
$1.5(n-1)$ ↓ half increasing and half decreasing	$(n-1)$ ↓ array is increasing
$2(n-1)$ ↓ decreasing array	

Note

[10, 2, 5, 6, 3, 1, 27, 9]
 0 1 2 3 4 5 6 7

max = 27
 min = 1

max = 27
 min = 1

max = 27
 min = 9

WITH DAC:

(10, 2, 5, 6 | 3, 1, 27, 9)

max = 10
 min = 2

(10 | 2 | 5 | 6)

max = 6
 min = 5

(3, 1, 27, 9)

max = 3
 min = 1

(27 | 9)

max = 10
 min = 2

(10 | 2)

max = 10
 min = 10

(10)

max = 2
 min = 2

(5)

max = 5
 min = 6

(6)

max = 6
 min = 6

3

max = 3
 min = 1

1

27

27

27

(9)

9

DAL(a, i, j)

for one element

```

{ if (i == j)
  max = min = a(i)
  return max, min
}

```

for two element

```

{ if (j == i + 1)
  { if (a(i) > a(j))
    { max = a(i)
      min = a(j)
    }
    else { max = a(j)
           min = a(i)
         }
    return (max, min)
  }
}

```

else

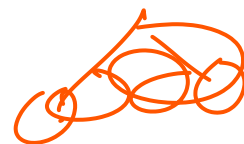
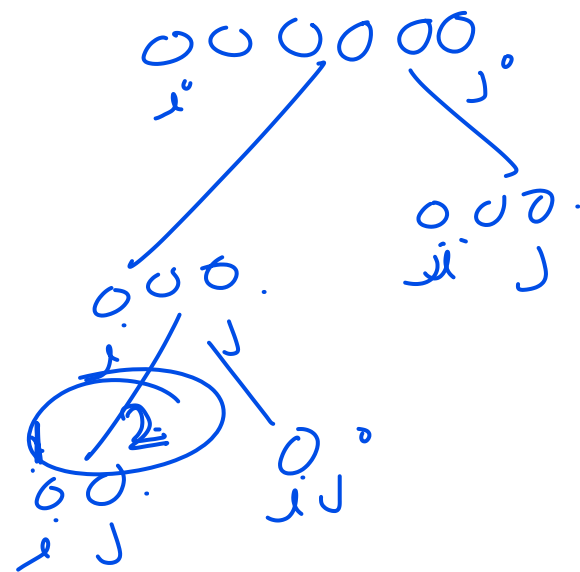
```

{ m = (i+j)/2
  max1, min1 = DAL(a, i, m)
  max2, min2 = DAL(a, m+1, j)
  if (max1 > max2)
    max = max1
  }

```

a = array
 i = starting
 j = ending

point = 0
 four = n-1



else { max = max2 }

if { min1 < min2 }

else { min = min1 }

{ min = min2 }

}

In vent le du
agars turn
um



Time Complexity:

